GITHUB ENGINEERING INTELLIGENCE SYSTEM

Analyser Architecture, Scorecard Design & Employer Usability

Version 5.0 · May 2026 · CTO Office · Engineering Hiring Intelligence

2 ANALYSIS MODES	7 PRIMITIVES	EVIDENCE BRIEF	ZERO SETUP
Light + Deep	Canonical Dimensions	No Composite Score	Light Mode Instant

1. Philosophy & Design Goals

The GitHub Engineering Intelligence System is an evidence-based analyser of observable engineering signals. It is designed as a force-multiplier for engineering leaders — not an automated hiring decision-maker. Its output is a structured Evidence Brief grounded in verifiable signals, honest about what it cannot observe, and organised around seven durable assessment primitives that map directly to on-the-job performance.

This document describes the full architecture of the analyser itself: how it collects data, what it measures, how it presents findings, and how companies interact with it. It does not cover downstream hiring workflows, offer management, or candidate onboarding — those processes remain with the hiring organisation.

1.1 Core Design Principles

Principle	What It Means in Practice
Evidence over scores	The primary output is an evidence brief — specific, cited, traceable — not a composite number. Scores are a summary of evidence, never a replacement for it.
Honest about uncertainty	Every assessed dimension carries a confidence level. Dimensions that cannot be assessed are surfaced explicitly as observability gaps, never as penalties.
Two analysis modes	A light public-signal mode requires no candidate action and starts instantly. A deep authenticated mode unlocks private signals with candidate consent. Both modes are meaningful on their own.
Leverage existing tools	Where mature open-source or third-party tools already solve a problem reliably, the system delegates to them rather than reimplementing. This reduces maintenance burden, accelerates development, and improves reliability.
Seven canonical primitives	All signals, layers, and output sections map back to one of seven durable assessment dimensions. Nothing that does not contribute to a primitive belongs in the system.
Frictionless for companies	Employers need no integration, no training, and no configuration to begin. Light Mode delivers an immediate brief from a GitHub

Principle	What It Means in Practice
	username alone.
Activity does not equal ability	GitHub signals are among the best available proxies for engineering behaviour — but they are proxies. The system never conflates observable GitHub output with engineering intelligence or potential.
AI use is a competency, not a red flag	Effective AI leverage is a primary engineering skill in 2026. The system classifies how a candidate uses AI — not whether they use it.

DESIGN MANDATE

Every signal added to the system must answer two questions: (1) Does it correlate with actual engineering performance, or with the appearance of performance? (2) What does it cost a motivated candidate to fake it, and does that cost exceed the benefit of gaming the system? Signals that fail either test are not included.

SYSTEM DESIGN SUMMARY

The GitHub Engineering Intelligence System produces a more rigorous, evidence-grounded, and uncertainty-honest assessment of an engineer's observable work history than any manual review could achieve in a reasonable time budget.

It does this through two analysis modes — a zero-friction Light Mode that works from a GitHub username alone, and a Deep Mode that unlocks private signals with candidate consent — both organised around seven durable canonical primitives that map directly to on-the-job engineering performance.

It delegates to mature external tools wherever they solve a problem better than a custom build would: GitHub's own search index for code similarity detection, third-party plagiarism APIs for laundering confirmation, an LLM API for all natural language analysis. It does not maintain custom indices, does not require HRIS integrations, and does not ask candidates to risk their current employment to participate.

Its output is an Evidence Brief — not a score — that tells hiring managers what was observed, how confident the system is in each conclusion, and precisely where human judgment must take over. Its governing principle is honesty: every observability gap is a limitation of the system, not a limitation of the candidate.

2. Two Analysis Modes

The system operates in two distinct modes that differ in access level, depth of analysis, and required setup. Both modes produce an Evidence Brief using the same seven-primitive framework; the difference is in the richness of signals available and the confidence levels achievable.

	Mode 1 — Light Analysis	Mode 2 — Deep Analysis
Trigger	Employer pastes a GitHub username or username list into the dashboard. No candidate action required.	Employer sends a candidate an evaluation link. Candidate authenticates via GitHub OAuth and installs the GitHub App on their account.
GitHub token	Platform system token — our shared GitHub App token. No per-candidate credential.	Candidate-specific installation token generated per GitHub App installation. Scoped to that candidate's repos and orgs.
Data access	Public repositories, public contribution activity, public PR/issue data, public profile, package registry presence, contribution calendar (public events only).	All public data plus: private repositories (with explicit consent), organisation membership and repos, full commit history, private PR review history, CI/CD run history, code scanning alerts, Dependabot alerts.
Local tool execution	Not performed. Analysis is API-only — no cloning. Faster, no compute overhead.	Full tool suite: scc, tokei, gitinspector, gitleaks, semgrep (lightweight ruleset). Top 30 repos cloned and analysed in parallel across 4 workers.
Primitives covered	All 7 primitives addressed at moderate-to-low confidence. Observability gaps are frequent and expected — surfaced explicitly.	All 7 primitives addressed. Higher confidence across the board. Fewer observability gaps. Private-work evidence significantly improves Authenticity Confidence.
Time to brief	Under 3 minutes for a typical public profile (50 repos, 5 years of history).	8–15 minutes depending on repository count and size. Runs asynchronously — employer is notified on completion.
Best used for	Initial screening of inbound applications. Rapid signal check before committing to a full evaluation or outreach.	Candidates who have passed initial screening and are being seriously considered. Pre-interview preparation. Full evidence gathering.
Rate limits	Platform token: 5,000 REST req/hr shared across all concurrent Light Mode runs. GraphQL: 5,000 points/hr. Search API: 30 req/min. Circuit breaker activates at <500 remaining.	Candidate installation token: 5,000 REST req/hr per candidate — fully isolated. No impact on Light Mode pool. GraphQL: 5,000 points/hr per installation.

IMPORTANT Both modes operate under read-only permissions at all times. No data is written to GitHub. No source code is stored beyond the analysis session. Only derived metrics and signals are persisted. Private repo code is analysed in-memory only during the Deep Mode session.

3. GitHub Access Configuration

3.1 Mode 1 — Platform System Token (Light Analysis)

Light Analysis uses a single GitHub App registered to the platform's own account. No installation by the candidate is required. The token is a standard GitHub App installation token issued for the platform's own installation, providing access to all public data on GitHub's API within normal rate limits.

Configuration Property	Value
Token type	GitHub App installation access token (JWT RS256 signed, auto-refreshed)
Scope	Public data only — no access to any private repositories or private organisation data
Rate limits	5,000 REST req/hr · 5,000 GraphQL points/hr · 30 Search req/min
Candidate action required	None — analysis triggered entirely by the employer
Token shared across	All concurrent Light Mode evaluations — budget managed with GraphQL-first strategy
Circuit breaker	Analysis pauses and caches partial results if remaining budget < 500 requests; resumes after rate limit window resets

3.2 Mode 2 — GitHub App Installation (Deep Analysis)

Deep Analysis requires the candidate to install the platform's GitHub App. This provides a per-candidate installation token scoped to exactly the repositories and organisations the candidate explicitly grants access to.

App Configuration	Detail
App type	GitHub App (not OAuth App, not Personal Access Token)
Installation target	Candidate's personal account + any organisation accounts they choose to share
Token strategy	Installation access tokens (1-hour lifetime, auto-refreshed via JWT RS256 signing)
Isolation	Each candidate's token is fully isolated — no shared rate limit pool with Light Mode
Consent flow	Candidate is shown the exact list of repositories that will be accessed before granting permission. Granular selection supported.
Data retention	Source code: analysed in-memory only, never stored. Derived metrics: retained for 90 days. Scores and Evidence Brief: retained for 12 months with candidate consent.
Compliance	GDPR/CCPA compliant. All raw API data purged within 30 days of collection.

Repository & Code Permissions (Deep Mode)

Permission	Access Level	Purpose
Contents	Read-only	Source code, commit history, file trees, test directory presence, CI config files

Permission	Access Level	Purpose
Metadata	Read-only (mandatory)	Repo stats, language breakdown, topics, fork/star counts, last push dates, archived status
Pull Requests	Read-only	PR descriptions, review comments, resolution time, diff size, approval patterns
Issues	Read-only	Issue quality, label discipline, closure ratios, triage behaviour
Commit Statuses	Read-only	CI pass/fail rates per commit, deployment frequency signals
Checks	Read-only	GitHub Actions results, linting outcomes, test run results
Code Scanning Alerts	Read-only	SAST findings — security discipline signals
Dependabot Alerts	Read-only	Dependency hygiene, CVE response time in their own projects
Deployments	Read-only	Deployment frequency, environment sophistication, rollback patterns
Organisation Members	Read-only	Org membership verification, team role confirmation (maintainer vs member)
Email addresses	Read-only	Identity verification, cross-platform correlation via commit email matching
GPG / SSH signing keys	Read-only	Commit signing rate — one component of security hygiene signal bundle only

4. Tooling Architecture — Existing Tools First

A core design principle of this system is to delegate to mature, maintained external tools and services wherever they already solve a problem reliably. This reduces the system's maintenance burden, avoids reimplementing solved problems, and allows engineering effort to focus on the analytical logic and output quality rather than infrastructure. The sections below describe each tool category and, critically, which custom-built solutions have been replaced by external alternatives.

4.1 Code Analysis Tools (Local Execution — Deep Mode Only)

Tool	Purpose	Why This Tool
scc (Sloc Cloc Code)	Ultra-fast codebase analyser. Reports SLOC, comment density, and complexity estimates per language. Detects auto-generated code bloat.	Rust binary, ~10ms per repo. Fastest available option with high accuracy.
tokei	Code counting per language including test file detection. Used to compute test-to-code ratios.	Faster than cloc, structured JSON output, excellent language support.
gitinspector	Per-author contribution statistics from git log: lines added/deleted, commits, days active. Isolates the candidate's actual contributions in multi-contributor repos.	Purpose-built for per-author analysis; more reliable than raw git log parsing.
gitleaks	Secret scanning against full git history — including past commits. Detects committed credentials even if later revoked.	Industry standard for this class of problem. Maintained, rule-updated regularly.
semgrep (OSS ruleset)	Lightweight SAST pattern matching against candidate code samples for common vulnerability classes.	Runs a focused ruleset locally without network dependency. Fast enough for screening-scale analysis.
github-linguist (npm)	Language detection identical to GitHub.com's own methodology. Excludes vendored and generated code from language attribution.	Same logic as GitHub — ensures language classifications in the brief match what the candidate sees on their own profile.
actionlint	Validates GitHub Actions workflow files for correctness and best-practice compliance.	Provides structured lint output that maps directly to CI/CD sophistication scoring.

4.2 Code Similarity & Laundering Detection — External Delegation

Earlier design iterations proposed building and maintaining a custom simhash and AST fingerprint index of the top 500,000 GitHub repositories to detect repository laundering. This approach has been deliberately rejected.

WHY WE DROPPED THE CUSTOM INDEX

Maintaining an AST and simhash index of 500,000 evolving codebases is computationally massive. Running real-time comparisons against this index during a live evaluation request would require a heavily decoupled asynchronous architecture with significant job queue infrastructure, aggressive caching, and a highly optimised database schema — just to evaluate a single candidate without timing out. The compute cost of the anti-gaming layer could exceed the compute cost of the actual evaluation. We delegate this problem to tools that have already solved it.

Tool / Service	Role	How It Substitutes the Custom Index
GitHub Code Search API	Primary laundering detection	GitHub's own search index already covers all public repositories. We query it with representative file signatures from the candidate's repos to detect near-duplicate matches across GitHub. No index to build or maintain — we use GitHub's existing one. Rate limit: 30 req/min authenticated.
Copyleaks Code API	Secondary plagiarism detection (optional)	Copyleaks offers a code-specific plagiarism detection API that compares submitted code against a maintained corpus including GitHub public repos, Bitbucket, and StackOverflow snippets. Used as a secondary confirmation for repos that trigger GitHub Search suspicion. Eliminates the need for custom fingerprinting infrastructure entirely.
LLM API (Claude)	NLP analysis & AI-generation detection	Commit message quality, PR description analysis, README evaluation, and AI-generation pattern detection are delegated to an LLM via API call. This replaces a custom NLP pipeline entirely. The LLM evaluates informativeness, intent communication, and structural patterns that indicate AI-generated content — at a quality level no custom pipeline would match.

4.3 External Signal APIs

API / Source	Signals Collected	Priority	Mode
npm Registry API	Weekly download counts, dependent package count, release cadence, deprecation history for any published npm packages by the candidate	Phase 2 — High value, hard to fake	Both
PyPI JSON API	Download statistics, dependent project counts, release history for any published Python packages	Phase 2 — High value	Both
Crates.io API	Download counts and dependent crates for any published Rust packages	Phase 2 — High value	Both
Stack Exchange public API	Reputation by technology tag, accepted answer rate, recency of activity. Tier 3 enrichment — additive only, absence carries no signal.	Phase 2 — Additive only	Both
GitHub Org Membership API	Cross-references stated employer organisations to confirm GitHub org membership — one component of the 3-rung employment verification ladder.	Phase 1 — Employment verification	Deep

STACK OVERFLOW NOTE

Stack Overflow participation has declined sharply since 2022 as developers route technical questions to LLMs instead. Stack Overflow reputation is retained as a Tier 3 additive signal only — high reputation in a relevant technology tag is a positive indicator when present, but zero Stack Overflow activity carries no negative weight. It is never used as a filter criterion or a scoring floor.

5. Data Collection

Data collection is structured into named groups. Each group feeds one or more of the seven canonical primitives. The table below shows which groups are collected in each analysis mode.

Group	Contents	Light	Deep
A — Identity & Profile	Public bio, company claim, linked blog/website, account age, hireable flag, commit email domains, GitHub org memberships (for employment verification)	✓	✓
B — Repository Inventory	All public repos: language breakdown, topics, README presence, last push dates, fork status (original vs. unmodified fork), homepage URLs, archived status. Deep adds private repos.	Public	Full
C — Commit Intelligence	Commit frequency distribution, message quality (via LLM API), commit size histograms, merge commit ratio, commit signing rate, work-hour distribution. Deep mode adds git clone + gitinspector + scc for top 30 repos.	API only	Full clone
D — Collaboration & Review	PR author/reviewer ratio, review comment depth, LGTM-only vs substantive review rate, PR size discipline, time-to-merge, self-merge rate, issue triage quality, cross-repo comment volume	Public PRs	Full (incl. org PRs)
E — Engineering Practices	File tree analysis (test dirs, CI configs, linting, Docker, IaC), Actions workflow sophistication, CI pass rates, release cadence, semantic versioning, SBOM depth, CVE response time, SAST density (semgrep), secret leak history (gitleaks)	File tree via API	Full incl. clone + tools
F — Impact & External Signals	External OSS contribution depth, contribution calendar, package registry signals (npm/PyPI/Cargo), Stack Overflow enrichment (Tier 3, additive)	Both modes	Both modes
G — Anti-Gaming Signals	Commit inflation patterns, fork dumping ratio, burst/dormancy fingerprinting, code similarity check via GitHub Code Search API + Copyleaks, AI-generation pattern detection via LLM API	Partial (API patterns)	Full (incl. clone analysis)

5.1 API Strategy & Rate Limit Management

The system applies a GraphQL-first strategy: nested and related data is fetched in single batched GraphQL queries wherever possible, reducing REST API consumption by approximately 60%. REST endpoints are used for flat data and file access. The Search API is used sparingly for cross-repository fingerprinting.

API Surface	Strategy	Savings
GraphQL API v4	Single batched query for contribution calendar, all PRs, all issue comments, external contributions. Handles all nested relational data.	~60% REST budget saved
REST API v3	Flat endpoints: repo list, commit stats, release history, file contents, CI run history. Paginated and parallelised.	Primary workhorse
Search API	Used only for cross-repo email-based commit discovery and code similarity spot-checks. Rate-limited to 30 req/min.	Used sparingly
Rate limit circuit breaker	If remaining budget < 500 requests: pause collection, cache partial results, and resume after reset window. Partial briefs never presented as complete.	Prevents API exhaustion

API Surface	Strategy	Savings
Repo prioritisation	For candidates with >100 repos: prioritise by (stars + forks + commit_count) × recency_weight. Evaluate top 50. This heuristic is disclosed in the brief.	Practical scale limit

6. The Seven Canonical Primitives

All signals, all analytical layers, and all output sections in this system map to one of seven durable assessment primitives. A primitive is a dimension that: (a) correlates with actual on-the-job engineering performance across contexts, (b) is measurable from observable signals, and (c) is stable enough to remain meaningful as tooling and practices evolve. Every signal that does not map to a primitive is excluded from the system.

Each primitive is assessed independently and presented with its own confidence level in the Evidence Brief. The output is never a composite of these seven into a single number — the hiring manager reads seven assessments, each bounded by explicit epistemic limits.

P1 Execution Reliability

Core Question: Can this engineer ship safely and consistently?

What it measures

- Commit pattern consistency (cadence, size discipline, co-authoring honesty)
- CI/CD pass rate trajectory over time
- Test-to-code ratio and test coverage trajectory
- Semantic versioning discipline in released software
- Dependency update hygiene — time to resolve Dependabot alerts
- Deployment frequency and rollback evidence

Signal sources: Group C (Commit Intelligence), Group E (Engineering Practices), tokei, scc, GitHub Actions API

P2 Systems Evolution

Core Question: Do systems improve under this engineer's stewardship over time?

What it measures

- Refactor commit quality and frequency
- Complexity trajectory over 2+ years of git history (measured via scc)
- API surface area stability — are abstractions getting cleaner?
- Migration safety patterns (backward compatibility, phased rollouts)
- Long-lived code survival — does their code outlast themselves?

Signal sources: Group C (Commit Intelligence), scc complexity trends, git history analysis, Group E

P3 Collaboration Leverage

Core Question: Does this engineer amplify the people around them?

What it measures

- PR review participation rate and substantive review rate (vs. LGTM-only)
- Review comment depth — do they identify root causes, not just symptoms?
- Evidence that their review comments lead to design changes
- Cross-repository engagement: issue triage,

external project contribution
- PR description quality — does the PR tell a story, explain trade-offs?
- Self-merge rate (inverted — high self-merge rate is a red flag at senior levels)

Signal sources: Group D (Collaboration & Review), GraphQL PR data, issue comment corpus, LLM-based depth scoring

Important qualification: Collaboration Leverage is the highest-value signal when present. When absent or thin, it carries no negative weight for candidates in enterprise, security, or embedded contexts where all review is private. The Evidence Brief distinguishes 'no review activity observed' from 'no review activity exists' and always defaults to the former interpretation.

P4 Technical Depth

Core Question: Can this engineer go deep when the problem genuinely requires it?

What it measures

- Depth in top 2–3 technologies by actual commit volume (not repo count)
- Evidence of handling genuinely hard problems: fault tolerance, concurrency, data consistency
- Operational maturity markers present in code: observability, retry logic, circuit breakers
- Package registry adoption — real-world usage of published packages by the community
- Review substance — do they identify architectural risks, not just style issues?
- Stack Overflow enrichment: accepted answers in relevant technical domains (Tier 3, additive)

Signal sources: Group B (Repo Inventory), Group D (Review analysis), LLM API analysis, Package Registry APIs, Stack Exchange API (Tier 3)

P5 Operational Maturity

Core Question: Can this engineer handle production reality?

What it measures

- Observability tooling presence: logging frameworks, metrics instrumentation, tracing setup
- Feature flag usage in production-facing code
- Migration safety: backward-compatible schema changes, phased rollout patterns
- Incident-response evidence: rollback commits, postmortem-referenced code changes
- Secret management hygiene — absence of credentials in git history (gitleaks)
- IaC presence for infrastructure-owning roles: Terraform, Pulumi, CDK

Signal sources: Group E (Engineering Practices), gitleaks, semgrep, Group C commit pattern analysis, IaC file detection

P6 AI Leverage Quality

Core Question: Can this engineer effectively direct AI to produce quality outcomes?

What it measures

- Velocity-to-quality ratio: high commit velocity periods with maintained or improving test coverage
- AI tool configuration files present: Cursor rules, .github/copilot-instructions.md, custom prompt files
- Evidence of AI-guided vs. AI-generated patterns: LLM-default code modified with architectural custom decisions
- Iterative refinement commits following large AI-assisted bursts — signals human review of AI output
- Style consistency: human code drifts stylistically; AI-assisted code with no human review shows abrupt discontinuities

Signal sources: Group C (Commit Intelligence), LLM API analysis, AI config file detection, commit velocity/quality correlation

AI Leverage Classification: candidates are classified across five patterns — AI Operator (high velocity, maintained quality), AI Architect (guiding AI rather than accepting output), AI Passenger (volume without judgment — risk flag), Traditional Engineer (consistent hand-crafted patterns — not penalised), and Disclosure Flag (AST entropy anomalies, style discontinuities — interview required to clarify, not automatic rejection).

P7 Authenticity Confidence

Core Question: Is the evidence trustworthy and the identity coherent?

What it measures

- Three-rung employment verification ladder (see Section 7.2)
- Gaming pattern detection: commit inflation, fork dumping, burst/dormancy fingerprinting
- Code similarity check via GitHub Search API and Copyleaks — laundering detection
- Authorship continuity: sudden sophistication jumps, style discontinuities, inconsistency between review voice and commit voice
- Credential leak history via gitleaks — hard security flag if detected

Signal sources: Group G (Anti-Gaming), gitleaks, GitHub Code Search API, Copyleaks Code API, LLM API for style analysis, Rung 1–3 verification

6.1 Seniority-Adjusted Primitive Weighting

Primitive weights are not fixed. They shift based on the target seniority level to reflect what actually matters at each career stage. A junior engineer is not penalised for lacking Collaboration Leverage or Operational Maturity signals that they have not yet had the opportunity to build. The table below shows relative weight emphasis — all seven primitives are always evaluated; only their contribution to the overall evidence narrative shifts.

Primitive	Intern / Junior	Mid-Level	Senior	Staff / Lead	Principal+
Execution Reliability	Primary	Primary	High	Moderate	Moderate
Systems Evolution	Not expected	Emerging	High	Primary	Primary
Collaboration Leverage	Minimal	Moderate	High	Primary	Primary
Technical Depth	High	High	High	High	High
Operational Maturity	Minimal	Moderate	High	High	Primary
AI Leverage Quality	Moderate	High	High	High	High
Authenticity Confidence	Always assessed equally regardless of seniority				

6.2 Role Archetype Signal Emphasis

Beyond seniority, certain signals within each primitive are elevated or contextualised based on the target engineering role. The system supports the following archetypes, with configurable signal emphasis per archetype:

Archetype	Elevated Signals	Contextual Red Flags
Backend Engineer	API design patterns in code, database migration files, performance tooling, profiling scripts, load testing configs. Key languages: Go, Rust, Java, Python, C++, Node.js.	No error handling patterns, no logging/instrumentation, no data layer tests
Frontend Engineer	TypeScript weighting above JavaScript, component library discipline, a11y configs, bundle size awareness, Storybook presence, visual regression tests.	No TypeScript, inline styles only, no test coverage, no accessibility attributes
Platform / DevOps / SRE	IaC (Terraform/Pulumi/CDK) presence, Kubernetes manifests, Helm charts, observability configs, GitOps patterns, secret management tooling.	Hardcoded credentials anywhere, no idempotency guards, bash without error handling
Data / ML Engineer	Notebook-to-pipeline transition evidence, data validation tooling, feature store usage, model versioning (MLflow/DVC), dbt configs, Airflow DAGs.	Notebooks only, no reproducibility tooling, no data validation, no productionisation
Security Engineer	CVE history management, responsible disclosure evidence (SECURITY.md), signed releases, SBOM generation in CI. Security signals amplified for penalty detection.	Any secret leak history, unpatched high/critical Dependabot alerts in own projects
Mobile Engineer	Xcode project structure quality, Gradle discipline, Fastlane presence, UI test frameworks (XCTest/Espresso), app store release evidence.	Hardcoded API keys in mobile code (critical), no UI tests, no signing configuration

7. Anti-Gaming Detection

GitHub profiles are increasingly optimised for hiring systems rather than reflecting genuine engineering activity. The detection layer is designed to increase the cost of gaming the system — not to catch every attempt, but to ensure that the effort required to successfully game the system exceeds the value gained. All gaming flags are surfaced as explicit findings in the Evidence Brief with evidence, confidence levels, and recommended interview questions. No flag produces an automatic rejection.

7.1 Detection Algorithms

Gaming Pattern	Detection Method	Threshold	Output
Commit inflation	Histogram of commit sizes (additions + deletions, excluding merges and doc-only commits). Flag if >30% of commits fall below 5-line threshold or p25 of commit size < 3 lines.	30% inflation rate	Technical Depth and Execution Reliability flags; contributes to Authenticity Confidence score
Fork dumping	Filter forks where gitinspector / contributor stats show zero commits by the candidate email. Flag if >50% of public repos are unmodified forks.	>50% unmodified forks	Repo inventory adjusted to exclude unmodified forks from language/topic analysis
Burst / dormancy pattern	Contribution heatmap analysis. Flag if last 30 days show >5× trailing 12-month weekly average — especially when evaluation was triggered recently.	>5× burst relative to 12-month trailing average	Consistency narrative note; interview question generated
Repository laundering	Representative file signatures from candidate repos queried against GitHub Code Search API. Repos with >40% near-duplicate file matches flagged. Copyleaks Code API used for secondary confirmation on flagged repos.	>40% file similarity to existing public repos	Authenticity Confidence flag; interview probe generated
AI-generation disclosure gap	LLM API analyses commit patterns, code style consistency, and structural entropy. Identifies abrupt style discontinuities correlated with large single-session commits, unnaturally exhaustive coverage patterns, and absence of iterative refinement.	Pattern confidence scored 0–100	Authenticity Confidence flag; classified on AI Leverage Quality scale; interview probe generated. Never automatic rejection.
Credential leak history	gitleaks run against all cloned repos including full git history, even for revoked secrets.	Any detection, any severity	Hard security flag — Operational Maturity score capped; escalated to hiring manager regardless of other scores. Cannot be cleared by the system — requires interview or background check.

7.2 Employment Verification — Three-Rung Ladder

Employment verification is treated as a confidence spectrum, not a binary check. The system communicates exactly what has and has not been verified at each rung. The 'Unverified' label is never used as a standalone output — it implies suspicion where the reality is simply insufficient data.

Rung	Mechanism	What It Actually Proves	Available In
Rung 1 — Email domain	Commit author email matches @employer.com domain	Candidate had access to a company email at some point. Weak signal. Does not prove scope or role.	Both modes
Rung 2 — Org membership	GitHub organisation membership API confirms presence in employer's GitHub org	Candidate had an active GitHub seat in the organisation. Stronger — org admins control this list directly.	Deep Mode
Rung 3 — Contribution fingerprint	Contribution activity in employer org repos is temporally consistent with the claimed employment tenure	Candidate was actively engineering at the claimed employer during the claimed period. Hard to fabricate retroactively.	Deep Mode

RUNG OUTPUT LANGUAGE

Each employment claim in the Evidence Brief displays the rung achieved. Example outputs: 'Rung 3 — Contribution fingerprint confirmed: active engineering activity in claimed organisation during stated period.' | 'Rung 1 only — email domain match. Contribution scope unconfirmed — recommend interview verification.' | 'Rung 0 — No verifiable signal available for claimed role. This is a system limitation, not a candidate failure. Proceed to interview with suggested probe.'

7.3 The Arms Race Limit

Static anti-gaming heuristics have a half-life measured in months. Once the detection mechanisms are known, motivated candidates will optimise against the detector. The long-term integrity of this layer depends not on technical sophistication alone, but on outcome correlation: the ability to demonstrate that candidates who trigger detection flags have meaningfully lower post-hire performance rates. Build the outcome data schema from Phase 1. The feedback loop is not just strategically valuable — it is what makes the anti-gaming layer epistemically defensible over time.

8. Evidence Brief — Analyser Output Format

The Evidence Brief is the primary output of the analyser. It is not a score, a ranking, or a recommendation — it is a structured account of what the system observed, what it concluded, how confident it is in each conclusion, and where human judgment must take over. The Brief is structured around the seven canonical primitives. Every claim is traceable to a specific signal, a specific data source, and a confidence level.

CRITICAL DESIGN PRINCIPLE

The system NEVER outputs a composite score. A number hides the evidence behind it, suppresses uncertainty, and invites decisions based on the number rather than the reasoning. The Evidence Brief is designed to make the hiring manager more informed, not to make a decision for them.

8.1 Brief Structure

Section	Content	Purpose
A — Profile in 90 Seconds	Engineering operating style archetype (Production Engineer / OSS Contributor / Generalist Builder / Specialist / Ops-Focused / Research-Oriented). Three strongest evidenced capabilities with specific citation. Employment verification rung achieved per claim. AI Leverage Quality classification. Analysis mode used (Light / Deep). Recommended interview depth.	Scannable by a recruiter or CTO in under a minute. Answers: who is this person, can I trust the evidence, how deep do we need to go?
B — Tech Reality vs. CV Claims	Languages: claimed vs. evidenced by actual commit volume over the last 3 years — not repo count, not self-reported. Frameworks: mentioned vs. demonstrated with meaningful usage depth in actual code. Infrastructure: claims vs. evidence of IaC, ops tooling, actual deployments. Explicit flags for CV claims with zero corroborating evidence.	Directly replaces the most-misrepresented section of any CV. Gives hiring managers a verifiable account before the first call.
C — Work Pattern Intelligence	Shipping velocity: time from issue creation to merged PR, averaged over career timeline. Quality discipline trajectory over time. Collaboration style: does the candidate seek review or avoid it? AI Leverage Quality classification with supporting evidence. Communication quality in PR descriptions and code comments.	Answers questions a CV structurally cannot answer: how does this person actually work, not just what have they built.
D — Red Flags & Verification Gaps	All gaming flags and verification gaps listed with: the specific evidence that triggered the flag, the confidence level (how certain is this a flag vs. a false positive), the recommended interview question to surface or clear the flag, and whether it is a hard stop or a soft concern requiring follow-up.	Gaming signals are never buried in score arithmetic. Every flag is visible, traceable, and actionable — with a clear path to resolution via interview.
E — Interview Intelligence	3–5 technical questions generated from actual design decisions observed in the candidate's code. Questions probing gaps between CV claims and evidenced capabilities. Questions that explore red flags without revealing the detection mechanism (allowing the candidate to either confirm or clear it). Suggested interviewer pairing recommendation.	Transforms the first technical conversation from generic assessment to targeted, evidence-grounded investigation.
F — Role & Stack Match	For Deep Mode and when a JD is provided: technical overlap between candidate's evidenced stack and the role requirements. Gap analysis: skills required by the role with no evidence in the	Tells the hiring manager specifically where to probe and what to not bother probing because evidence is already

Section	Content	Purpose
	candidate's profile — translated into specific interview probe topics. JD intent extraction: what the role actually needs, beyond keyword matching.	strong.
G — What This Evaluation Cannot Tell You	Explicit list of the qualities within the epistemic boundary of the system — routed to specific interview probes. This section is permanent and never omitted. It tells the hiring manager exactly where human judgment must take over.	Trust builder. Makes the system more credible by being honest about its limits before someone discovers them.

8.2 Confidence Levels — Mandatory Language

Every assessed dimension in the Evidence Brief carries a confidence level. The following language is mandatory — implementers may not substitute different formulations, as variance in confidence language leads to misinterpretation.

Level	Trigger Condition	Mandatory Output Language
Strong Evidence	3 or more independent signals confirming the same capability across 12 or more months of activity	"Demonstrated across [N] repositories and [N] months — high confidence."
Moderate Evidence	1–2 signals or a single time window of evidence	"Evidenced in limited context — probe in interview to confirm depth."
Low Evidence	Single weak signal or isolated instance	"One instance detected — insufficient to score. Treat as unconfirmed in hiring decision."
Observability Gap	Signal is expected for a candidate of this seniority or role, but is absent or unverifiable (likely due to private enterprise work, classified context, or a non-GitHub workflow)	"No public evidence — likely private or enterprise context. Do not penalise. Recommend: [specific interview question to probe the capability directly]."
Insufficient Data (Profile-Level)	Majority of primary primitives return Observability Gap — typically senior enterprise or regulated-industry engineers	"This profile cannot be assessed from available public signals. Do not use this report as a filter for this candidate. Proceed directly to technical interview using the generated interview questions."

PROFILE-LEVEL GATE

When a senior-level candidate (Staff or above based on CV claims) produces a profile-level Insufficient Data output, the Evidence Brief must include: 'This profile pattern is consistent with enterprise or regulated-industry engineering contexts where public evidence is structurally absent. This is correlated with — not anticorrelated with — seniority and impact. Proceed to technical interview.'

11. System Pipeline Architecture

11.1 Light Mode Pipeline

Step	Action	Target Time
1 — Trigger	Employer submits username(s) and role configuration. Job queued.	< 1 second
2 — Public data crawl	REST + GraphQL fetch of Groups A, B (public), C (API-only), D (public), F. Rate limit budget tracked.	~60 seconds
3 — External signals	Package registry lookups (npm/PyPI/Cargo). Stack Overflow enrichment (if present).	~15 seconds
4 — Anti-gaming analysis	Commit pattern analysis via API data. GitHub Code Search for code similarity spot-checks on flagged repos.	~30 seconds
5 — LLM analysis	Commit message quality, PR description depth, AI-generation pattern scoring via LLM API batch call.	~30 seconds
6 — Evidence Brief generation	Seven-primitive assessment with confidence levels. Employment verification rungs computed. Brief structured and rendered.	~15 seconds
7 — Delivery	Brief available in dashboard. Employer notified.	< 3 minutes total

11.2 Deep Mode Pipeline

Step	Action	Target Time
1 — Evaluation link	Employer generates evaluation link. Candidate receives it and installs GitHub App, selecting repositories and orgs to include.	Async — candidate-paced
2 — Token generation	Installation ID received. Installation access token generated (JWT RS256). Isolated rate limit pool initialised.	< 5 seconds
3 — Inventory crawl	All repos (owned + contributed to). Rate limit budget calculated per candidate based on repo count. Top 30 repos selected by quality signal.	~60 seconds
4 — GraphQL bulk fetch	Single query: contribution calendar, all PRs, all issue comments, external contributions, org memberships. Saves ~60% of REST budget.	~30 seconds
5 — Deep repo analysis	Top 30 repos cloned via HTTPS (token-authenticated). scc + tokei + gitinspector + gitleaks + semgrep run in parallel across 4 workers.	~5–8 minutes
6 — External signals	Package registry APIs. Stack Overflow enrichment. Employment verification rungs 1–3.	~30 seconds
7 — Anti-gaming analysis	Full pattern analysis on clone data. GitHub Code Search + Copyleaks for laundering detection on flagged repos.	~2 minutes
8 — LLM analysis	Full commit history analysis, PR description corpus, README scoring, AI-generation pattern detection.	~60 seconds
9 — Scoring & brief generation	Seven-primitive assessment. Confidence thresholds applied. Profile-level sufficiency gate checked. Evidence Brief structured and rendered.	~30 seconds
10 — Delivery	Employer notified. Brief available in dashboard.	8–15 minutes total